Name: Khushi Chhaproo
Div.: D20C
Roll No.: 07

Experiment - 01
Cryptography in Blockchain – Merkle Root Tree Hash

Aim:
To explore cryptographic concepts used in Blockchain and implement SHA-256 hashing, nonce-based cryptographic puzzle solving, and Merkle Tree Root Hash construction using Python.

Tasks Performed:

The following tasks were performed during the experiment:

1. Implemented a Python program to generate a **SHA-256 cryptographic hash** using the `hashlib` library.
2. Generated a hash by combining an input string with a **nonce** value.
3. Implemented a **cryptographic puzzle** to find a nonce that produces a hash beginning with a specified number of leading zeros.
4. Constructed a **Merkle Tree** using a given set of blockchain transactions.
5. Used the following transactions for Merkle Tree construction:
    - T1: Alice → Bob : $200
    - T2: Bob → Dave : $500
    - T3: Dave → Eve : $100
    - T4: Eve → Alice : $300
    - T5: Roo → Bob : $50
6. Hashed the transactions before combining them to construct the Merkle Tree.
7. Printed the intermediate hashes generated at each level of the Merkle Tree.
8. Generated the final **Merkle Root Hash**.
9. Included the student name **Khushi Chhhaproo** in the data used for hashing.

These tasks follow the four activities specified in the lab material: SHA-256 hash generation, nonce-based hashing, Proof-of-Work puzzle solving, and Merkle Tree construction.

Programs:

Blockchain Lab 1 - Khushi Chhaproo - D20C 07

Program 1 — SHA-256 Hash Generation

```
import hashlib


# Student name

student_name = "Khushi Chhaproo"


def create_hash(string):

    # Create a hash object using SHA-256 algorithm

    hash_object = hashlib.sha256()


    # Convert the string to bytes and update the hash object

    hash_object.update(string.encode('utf-8'))


    # Get the hexadecimal representation of the hash

    hash_string = hash_object.hexdigest()


    # Return the hash string

    return hash_string
```

```
# Use student's name as input

input_string = student_name


# Generate hash

hash_result = create_hash(input_string)


# Display output

print("Student Name:", student_name)

print("Input String:", input_string)

print("SHA-256 Hash:", hash_result)
#error statement
```

Output:

```
Student Name: Khushi Chhaproo
Input String: Khushi Chhaproo
SHA-256 Hash: 3d2c0b743082d4f03b46bc4fbde636b164c850d83f64688b320c331fc87f91a8
```

Program 2 — Target Hash Generation Using Nonce

```
import hashlib


# Student name

student_name = "Khushi Chhaproo"


# Input string

input_string = student_name
```

```
# Nonce value

nonce = 10


# Concatenate the string and nonce

hash_string = input_string + str(nonce)


# Calculate the hash using SHA-256

hash_object = hashlib.sha256(hash_string.encode('utf-8'))

hash_code = hash_object.hexdigest()


# Print the result

print("Student Name:", student_name)

print("Input String:", input_string)

print("Nonce:", nonce)

print("String Hashed:", hash_string)

print("Hash Code:", hash_code)                                          #
Error
```

Output:

```
Student Name: Khushi Chhaproo
Input String: Khushi Chhaproo
Nonce: 10
String Hashed: Khushi Chhaproo10
Hash Code: c2108661f2ed60b9374b630e97039866634f16a9add982c71138982a3fdce09e
```

Program 3 — Cryptographic Puzzle / Golden Nonce

```
import hashlib


# Student name

student_name = "Khushi Chhaproo"




def find_nonce(input_string, num_zeros):


    nonce = 0


    # Create the required hash prefix

    hash_prefix = '0' * num_zeros


    while True:


        # Concatenate input string and nonce

        hash_string = input_string + str(nonce)


        # Calculate SHA-256 hash

        hash_object = hashlib.sha256(hash_string.encode('utf-8'))

        hash_code = hash_object.hexdigest()
```

```
        # Check if hash starts with required number of zeros
        if hash_code.startswith(hash_prefix):

            print("Hash:", hash_code)

            return nonce

        # Try next nonce
        nonce += 1



# Input string
input_string = student_name

# Number of leading zeros required
num_zeros = 1

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the result
print("Student Name:", student_name)
print("Input String:", input_string)
```

```
print("Leading Zeros:", num_zeros)

print("Golden Nonce:", expected_nonce)
```

Output:

```
Hash: 00db982533f4d0c6e264bed24f7d9611668961632a4698dcdb18868fa54c66ee
Student Name: Khushi Chhaproo
Input String: Khushi Chhaproo
Leading Zeros: 1
Golden Nonce: 11
```

Program 4 — Merkle Tree Root Hash

```
import hashlib


# Student name

student_name = "Khushi Chhaproo"




# ------------------------------------------------

# FUNCTION TO CALCULATE SHA-256 HASH

# ------------------------------------------------


def calculate_hash(data):

    return hashlib.sha256(data.encode('utf-8')).hexdigest()
```

```python
# ------------------------------------------------

# FUNCTION TO BUILD MERKLE TREE

# ------------------------------------------------

def build_merkle_tree(transactions):

    # If there are no transactions

    if len(transactions) == 0:

        return None

    # ------------------------------------------------

    # STEP 1: HASH EACH TRANSACTION

    # ------------------------------------------------

    hashed_transactions = []

    print("\n========== TRANSACTION HASHES ==========")

    for transaction in transactions:

        # Include student's name in the data being hashed

        data_to_hash = student_name + " | " + transaction
```

```
    # Calculate transaction hash
    transaction_hash = calculate_hash(data_to_hash)

    # Store hash
    hashed_transactions.append(transaction_hash)

    # Print transaction information
    print("Transaction:", transaction)
    print("Data Hashed:", data_to_hash)
    print("Hash:", transaction_hash)
    print()



# ------------------------------------------------
# STEP 2: BUILD MERKLE TREE
# ------------------------------------------------

level = 1
current_level = hashed_transactions

while len(current_level) > 1:

    print("========== MERKLE TREE LEVEL", level, "==========")
```

```
# If the number of hashes is odd,
# duplicate the last hash
if len(current_level) % 2 != 0:

    current_level.append(current_level[-1])

    print("Odd number of hashes detected.")
    print("Last hash duplicated.")
    print()

# Store hashes for next level
new_level = []

# Process two hashes at a time
for i in range(0, len(current_level), 2):

    # Get left and right hashes
    left_hash = current_level[i]
    right_hash = current_level[i + 1]

    print("Left Hash :", left_hash)
    print("Right Hash:", right_hash)
```

```
            # Combine the two hashes
            combined = left_hash + right_hash

            print("Combined Hashes:", combined)

            # Hash the combined value
            parent_hash = calculate_hash(combined)

            # Store parent hash
            new_level.append(parent_hash)

            print("Parent Hash:", parent_hash)
            print()

        # Move to the next level
        current_level = new_level

        level += 1


# Final hash is the Merkle Root
return current_level[0]
```

```
# ------------------------------------------------

# TRANSACTIONS

# ------------------------------------------------


transactions = [

    "Alice -> Bob : $200",

    "Bob -> Dave : $500",

    "Dave -> Eve : $100",

    "Eve -> Alice : $300",

    "Roo -> Bob : $50"

]




# ------------------------------------------------

# DISPLAY TRANSACTIONS

# ------------------------------------------------


print("==========================================")

print("       MERKLE TREE CONSTRUCTION")

print("==========================================")
```

```
print("Student Name:", student_name)


print("\nTransactions:")


for i, transaction in enumerate(transactions, start=1):

    print("T" + str(i) + ":", transaction)




# ------------------------------------------------

# BUILD MERKLE TREE

# ------------------------------------------------


merkle_root = build_merkle_tree(transactions)




# ------------------------------------------------

# DISPLAY FINAL MERKLE ROOT

# ------------------------------------------------


print("\n============================================")

print("             FINAL RESULT")

print("============================================")
```

```
print("Student Name:", student_name)

print("Merkle Root Hash:", merkle_root)
```

Output:

```
    Transaction: Dave -> Eve : $100
    Data Hashed: Khushi Chhaproo | Dave -> Eve : $100
    Hash: 48612b01d4818ad29c5b30c1bbf62a5c3759bc9ab0d0917ea8b114f349717bb6

    Transaction: Eve -> Alice : $300
    Data Hashed: Khushi Chhaproo | Eve -> Alice : $300
    Hash: 4af0b866d4776f5989f50a166c91365e93ab3b1170986af0d0dfbf1b8b1fdebc

    Transaction: Roo -> Bob : $50
    Data Hashed: Khushi Chhaproo | Roo -> Bob : $50
    Hash: 99dae37ce9fb39516306dcc98707ed33cb5c17866b8415373c66606116946172

    ========== MERKLE TREE LEVEL 1 ==========
    Odd number of hashes detected.
    Last hash duplicated.

    Left Hash : 238842e6e045016788ff6fcbbe61ebf4314888818ac82c7c11aa5708033f050c
    Right Hash: 2f3fdc71d11a163003d2de30d1c6d055abc9982239325969fb3f5b79a3d3d246
    Combined Hashes: 238842e6e045016788ff6fcbbe61ebf4314888818ac82c7c11aa5708033f050c2f3fdc71d11a163003d2de30d1c6d055abc9982239325969fb3f5b79a3d3d246
    Parent Hash: 5e344d7bdb0eb0cc94fe4d5e3aff372fd1b43488c7e117b897d97e9aadfb54c5

    Left Hash : 48612b01d4818ad29c5b30c1bbf62a5c3759bc9ab0d0917ea8b114f349717bb6
    Right Hash: 4af0b866d4776f5989f50a166c91365e93ab3b1170986af0d0dfbf1b8b1fdebc
    Combined Hashes: 48612b01d4818ad29c5b30c1bbf62a5c3759bc9ab0d0917ea8b114f349717bb64af0b866d4776f5989f50a166c91365e93ab3b1170986af0d0dfbf1b8b1fdebc
    Parent Hash: 1dfb3d6bd2298f678f0c25c4f2594bad298da81edbd1d64e53c9f3ab389479e7
```

Activate Windows

```
Left Hash : 99dae37ce9fb39516306dcc98707ed33cb5c17866b8415373c66606116946172
Right Hash: 99dae37ce9fb39516306dcc98707ed33cb5c17866b8415373c66606116946172
Combined Hashes: 99dae37ce9fb39516306dcc98707ed33cb5c17866b8415373c666061169461729dae37ce9fb39516306dcc98707ed33cb5c17866b8415373c66606116946172
Parent Hash: cb286c5a5890c2ebdeec96c8058a1eedee02a43d0e9f0a00beb70eb50e15cb99

========== MERKLE TREE LEVEL 2 ==========
Odd number of hashes detected.
Last hash duplicated.

Left Hash : 5e344d7bdb0eb0cc94fe4d5e3aff372fd1b43488c7e117b897d97e9aadfb54c5
Right Hash: 1dfb3d6bd2298f678f0c25c4f2594bad298da81edbd1d64e53c9f3ab389479e7
Combined Hashes: 5e344d7bdb0eb0cc94fe4d5e3aff372fd1b43488c7e117b897d97e9aadfb54c51dfb3d6bd2298f678f0c25c4f2594bad298da81edbd1d64e53c9f3ab389479e7
Parent Hash: 190c4bdc462c4da0a4ec1778acb6a94fbe677ae6dec95bfdf80e11fe06b9caeb

Left Hash : cb286c5a5890c2ebdeec96c8058a1eedee02a43d0e9f0a00beb70eb50e15cb99
Right Hash: cb286c5a5890c2ebdeec96c8058a1eedee02a43d0e9f0a00beb70eb50e15cb99
Combined Hashes: cb286c5a5890c2ebdeec96c8058a1eedee02a43d0e9f0a00beb70eb50e15cb99cb286c5a5890c2ebdeec96c8058a1eedee02a43d0e9f0a00beb70eb50e15cb99
Parent Hash: 186d15927f6bf2e7d245b65f39383d807882d50ac6a3b11e9fbe84157bc30351

========== MERKLE TREE LEVEL 3 ==========
Left Hash : 190c4bdc462c4da0a4ec1778acb6a94fbe677ae6dec95bfdf80e11fe06b9caeb
Right Hash: 186d15927f6bf2e7d245b65f39383d807882d50ac6a3b11e9fbe84157bc30351
Combined Hashes: 190c4bdc462c4da0a4ec1778acb6a94fbe677ae6dec95bfdf80e11fe06b9caeb186d15927f6bf2e7d245b65f39383d807882d50ac6a3b11e9fbe84157bc30351
Parent Hash: 4b12f2be991ee7429c7969b339221b450f5ddb2a72fe8b3ca7786d48e656b0fe
```

Activate Windows

```
==========================================
              FINAL RESULT
==========================================
Student Name: Khushi Chhaproo
Merkle Root Hash: 4b12f2be991ee7429c7969b339221b450f5ddb2a72fe8b3ca7786d48e656b0fe
```

Conclusion:

The experiment successfully demonstrated the fundamental cryptographic concepts used in blockchain technology. SHA-256 hashing was implemented to generate secure hash values, while the nonce-based cryptographic puzzle demonstrated the concept of Proof-of-Work by finding a nonce that satisfies a required hash condition. A Merkle Tree was constructed by hashing the given transactions and repeatedly combining the hashes to generate a single Merkle Root Hash. Thus, the experiment provided an understanding of hashing, nonce, Proof-of-Work, Merkle Trees, and their importance in maintaining data integrity and efficient verification in blockchain systems.